Quizz 6 - Graphes

Durée: 25min **2025/2026**

L'utilisation de toute calculatrice, de tout matériel électronique et de tout document est interdite

1. Question de cours

Q1.
Répondre aux questions suivantes sur le graphe 1 :

- Orienté : ◻ Oui ◻ Non
- Pondéré : ◻ Oui ◻ Non
- Connexe : ◻ Oui ◻ Non
- Degré du sommet D : _____
- Nombre de composantes connexes : _____
- Nombre de cycles : _____

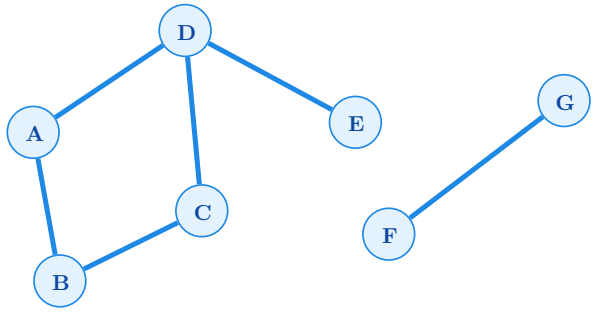

Fig. 1. – Graphe 1

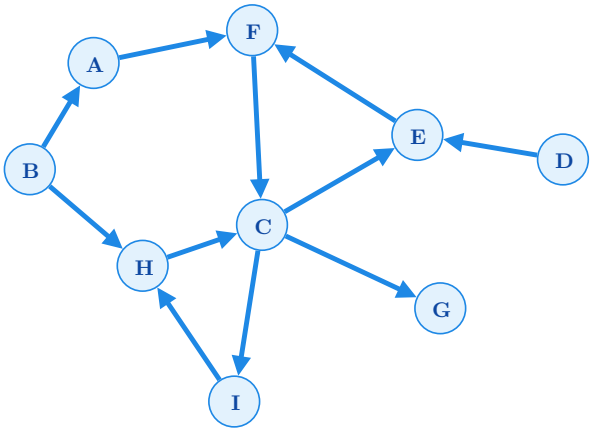

Fig. 2. – Graphe 2

Q2.
Répondre aux questions suivantes sur le graphe 2 :

- Orienté : ◻ Oui ◻ Non
- Pondéré : ◻ Oui ◻ Non
- Fortement connexe : ◻ Oui ◻ Non
- Degrés du sommet C : _____
- Nombre de cycles : _____
- F est-il accessible depuis I ? ◻ Oui ◻ Non
- A est-il accessible depuis I ? ◻ Oui ◻ Non

2. Représentations et fonctions de base

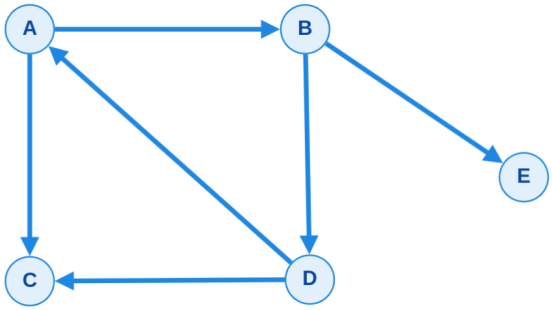

Fig. 3. – Graphe 3

Q3.
Donnez la matrice d'adjacence du **Graphe 3** sous forme de liste de listes Python.

Q4.
Donnez la liste d'adjacence du **Graphe 3** sous forme de dictionnaire Python.

Pour les fonctions suivantes, on considère que le graphe est représenté par une liste d'adjacence sous forme de dictionnaire Python.

Q5.
Écrire une fonction `est_successeur(G: dict, u: str, v: str) -> bool` qui vérifie si un sommet v est un successeur du sommet u dans G (donc si un arc `u` $\rightarrow$ `v` existe).

Q6.
Écrire une fonction `est_predecesseur(G: dict, u: str, v: str) -> bool` qui vérifie si un sommet v est un prédécesseur du sommet u dans G (donc si un arc `v` $\rightarrow$ `u` existe).

Q7.
Écrire une fonction `degres_sortant(G: dict, u: str) -> int` qui retourne le degré sortant du sommet u dans G.

Q8.
Écrire une fonction `degres_entrant(G: dict, u: str) -> int` qui retourne le degré entrant du sommet u dans G. **Donnez la complexité de votre fonction.**

3. Parcours

Q9.
Compléter la fonction `est_accessible(G: dict, src: str, dest: str) -> bool` qui vérifie si un sommet v est accessible depuis le sommet u dans G. Vous implémenterez cette fonction sur une base du **parcours en profondeur**.

```
1   def est_accessible(G, src, dest):
2       # initialisation d'une pile et d'un ensemble de sommets visités
3       visites = set()
4       pile = [src]
5       while ___________________:
6           sommet = ___________________
7           if sommet not in visites:
8               # on marque sommet comme visité et on ajoute ses successeurs à la pile
9               visites.add(sommet)
10              for successeur in ______________________:
11                  if ________________________:
12                      return True
13                  elif successeur not in visites:
14                      ________________________________
15      return False
```

Q10.
Compléter la fonction `taille_plus_court_chemin(G: dict, src: str, dest: str) -> int` qui retourne la taille du plus court chemin entre les sommets `src` et `dest` dans G.

```
def taille_plus_court_chemin(G, src, dest):
    visites = {src}
    # initialisation de la file
    file = ______________________

    while ____________________:
        # On défile le premier élément ajouté
        sommet, dist = ___________________________

        # On ajoute les successeurs non visités à la file
        for successeur in _________________:
            if ____________________________ :
                return ____________________________
            if successeur not in visites:
                visites.add(successeur)
                ____________________________________
    return None
```

4. Dijkstra

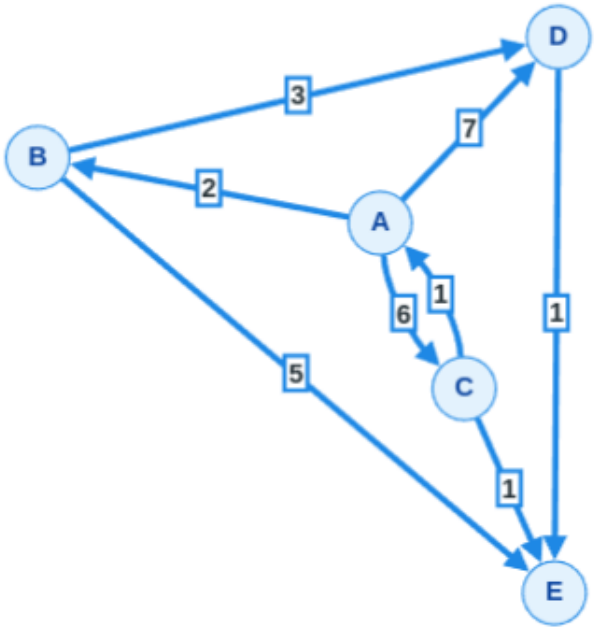

Fig. 4. – Graphe 4

Q11.
Dépilez l'algorithme de Dijkstra sur le **Graphe 4** en partant du sommet `A` en complétant le tableau suivant pour chaque tour de boucle.

Action	**A**	**B**	**C**	**D**	**E**	**`a_visiter`**
Initialisation	0	∞	∞	∞	∞	`{A, B, C, D, E}`